

MACHINE LEARNING

Feature Scaling

A Complete Guide to Scaling, Normalization & Best Practices



Presented by Nancy Ahmed

Data Science & Machine Learning

Session Overview



Intuition & Why It Matters

1



Types of Scaling (MinMax, Standard, Robust)

2



Python Implementation

3



Common Mistakes & Data Leakage

4



Pipelines & Best Practices

5



Interview Questions

6

What is Feature Scaling?



Transforming numerical features into a similar range or distribution so that no single feature dominates the model.

The Problem Without Scaling:

Feature	Range	Impact
Age	18 – 60	Small — low influence
Salary	10,000 – 500,000	Huge — DOMINATES the model!



Salary dominates Euclidean distance — the model ignores Age completely!



Algorithms That REQUIRE Scaling



Distance-Based

- K-Nearest Neighbors (KNN)
- K-Means Clustering
- Support Vector Machine (SVM)



Distance calculations are distorted by large-scale features



Gradient-Based

- Logistic Regression
- Linear Regression
- Neural Networks



Uneven gradients slow convergence & hurt optimization

Algorithms That DON'T Need Scaling



Tree-Based Models

- Decision Trees
- Random Forest
- XGBoost
- LightGBM
- Gradient Boosting

Why Trees Don't Need Scaling



Threshold-Based Splits

Trees split data using threshold values (e.g., Age > 30), not distances. Scale doesn't affect which split is best.



Feature Independence

Each feature is evaluated independently when finding the best split point — no comparison between feature magnitudes.



Order-Preserving

Only the relative order of values matters for tree splits, not their absolute magnitude or scale.

★ Common Interview Question — Know This!

Normalization — Min-Max Scaling

Formula

$$x_{\text{scaled}} = (x - x_{\text{min}}) \div (x_{\text{max}} - x_{\text{min}})$$

Original	Scaled [0, 1]
10 (min)	0.0
20	0.5
30 (max)	1.0



Advantages

- Keeps values in [0, 1] — bounded range
- Ideal for Neural Networks
- Preserves the shape of the distribution

Python

```
from sklearn.preprocessing import  
MinMaxScaler  
  
scaler = MinMaxScaler()  
  
# Fit and transform training data  
X_train_scaled =  
scaler.fit_transform(X_train)  
  
# Only transform test data  
X_test_scaled = scaler.transform(X_test)
```



Disadvantage

- Very sensitive to outliers!
- One huge outlier (e.g. 1000) compresses all other values into a tiny range

★ Standardization — Z-Score Scaling (Most Important!)

Formula

$$z = (x - \mu) \div \sigma$$

μ

Mean of the feature

σ

Standard deviation

z

Standardized value

Mean after scaling = 0

Std Dev after scaling = 1

Python

```
from sklearn.preprocessing import
StandardScaler

scaler = StandardScaler()

# Fit on train, transform both
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Check results
print(X_train_scaled.mean()) # → 0.0
print(X_train_scaled.std())  # → 1.0
```



Advantages

- Works well for most ML models
- Handles normally distributed data well
- Most commonly used scaler in practice

Still moderately affected by extreme outliers

Robust Scaling — Outlier-Resistant



Use this scaler when your dataset has many outliers — it uses median & IQR instead of mean & std.

Formula

$$x_{\text{scaled}} = (x - \text{Median}) \div \text{IQR}$$

IQR = Q3 - Q1 (Interquartile Range)

Python

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Outlier Example:

`data = [1, 2, 3, 4, 1000]` $\leftarrow$ 1000 is a massive outlier

MinMaxScaler

All normal values compressed to ~0.0–0.004 ✗ BAD

RobustScaler

Normal values spread well, outlier just shifted ✓ GOOD

Which Scaler Should You Use?

Aspect	MinMax (Normalization)	StandardScaler	RobustScaler	MaxAbsScaler
Range Output	[0, 1]	No fixed range	Centered around 0	[-1, 1]
Outlier Sensitivity	Very High	Moderate	Resistant	Moderate
Distribution	Preserved	Mean=0, Std=1	Median-centered	Max Abs = 1
Best For	Neural Networks	General ML	Data with outliers	Sparse data
Uses Mean/Std	No	Yes	No (Median/IQR)	No



Neural Networks
→ MinMaxScaler



General ML
→ StandardScaler



Many Outliers
→ RobustScaler



Sparse Data
→ MaxAbsScaler



⚠️ DATA LEAKAGE — The #1 Critical Mistake!

✗ WRONG WAY

```
# Dangerous! Never do this:
scaler = StandardScaler()

# Fitting on ALL data before split
X_scaled = scaler.fit_transform(X)

X_train, X_test = train_test_split(
    X_scaled
)
# Test data info leaked into scaler!
```

Test statistics leak into training — model appears better than it is!



DO
THIS
INSTEAD

✓ CORRECT WAY

```
# Safe: Split FIRST, then scale
X_train, X_test = train_test_split(
    X, test_size=0.2
)

scaler = StandardScaler()

# Fit ONLY on training data
X_train_scaled = scaler.fit_transform(
    X_train
)
# Apply same scaler to test
X_test_scaled = scaler.transform(
    X_test
)
```

fit() vs transform() vs fit_transform()



fit()

Learns from the data

Calculates and stores statistics:

- Mean (μ)
- Standard deviation (σ)
- Min / Max values

Does NOT change the data!

Use on: Training data only



transform()

Applies scaling

Uses the learned statistics to scale the data.

Does NOT re-learn!
Uses previously fitted values.

Use on: Both train & test data



fit_transform()

Learns AND applies

Combines fit() + transform() in one step.

Convenient shortcut.
Slightly more efficient.

Use on: Training data only!

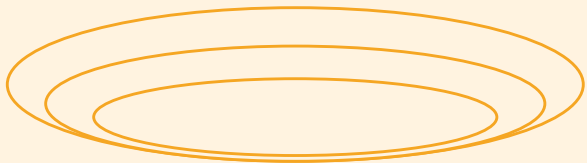
Correct ML Workflow with Feature Scaling



💡 Tip: Use sklearn Pipelines to automate this workflow and prevent leakage automatically!

Scaling & Gradient Descent — Why Speed Matters

✗ Without Scaling

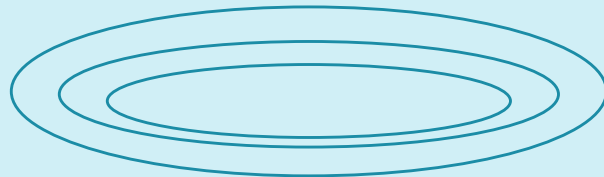


*Elongated contours →
Zigzag path to minimum
= Slow convergence*

- Large gradient steps on big features
- Small gradient steps on tiny features
- Oscillation & slow convergence
- May need many more epochs

VS

✓ With Scaling



*Circular contours →
Direct path to minimum
= Fast convergence*

- Equal gradient scale for all features
- Faster and smoother convergence
- Fewer training epochs needed
- Better final model performance



Common Mistakes

01



✗ **Scaling BEFORE train-test split**

✓ Always split first, then fit scaler on training data only

02



✗ **Using MinMaxScaler with outliers**

✓ Use RobustScaler when your data has outliers

03



✗ **Scaling the target variable (y)**

✓ Never scale y unless your specific task requires it (e.g., regression output inverse transform)

04



✗ **Forgetting to scale test data**

✓ Always apply the same `scaler.transform()` to test data

05



✗ **Re-fitting scaler on test data**

✓ `fit()` only on training data — never call `fit_transform()` on test data



Interview Questions — Know These!

1 Why do we scale features?

To prevent large-scale features from dominating distance/gradient calculations, ensuring fair contribution from all features.

2 Which algorithms need scaling?

Distance-based (KNN, SVM, K-Means) and gradient-based (Linear/Logistic Regression, Neural Networks, PCA).

3 Why don't tree-based models need scaling?

Trees split on thresholds — they're order-based, not distance-based, so scale doesn't affect split quality.

4 Normalization vs Standardization?

MinMax $\rightarrow [0,1]$, sensitive to outliers. StandardScaler $\rightarrow$ mean=0, std=1, works for most models.

5 What is data leakage in scaling?

Fitting the scaler on all data before splitting — test statistics leak into training, giving falsely optimistic results.

6 Why scale AFTER train-test split?

To simulate real-world deployment where we never see test data during training.

Key Takeaways



Scaling doesn't make your model smarter — it makes it learn more efficiently



Always fit the scaler on training data only — never on test data



Tree-based models (Decision Tree, Random Forest, XGBoost) don't need scaling



Use RobustScaler when your data has outliers



Use sklearn Pipelines in production to avoid leakage and simplify code



Always scale before PCA and Neural Networks for best performance



Thank You!

*"Feature scaling does not change the intelligence of the model
— it changes how efficiently the model learns."*

Presented by Nancy Ahmed

Machine Learning & Data Science